

Understanding Computer Architecture

Name - Saikat Sheet

Batch - 25SUB5936 Linux System Programming

ID - 68500

Overview of Computer Architecture

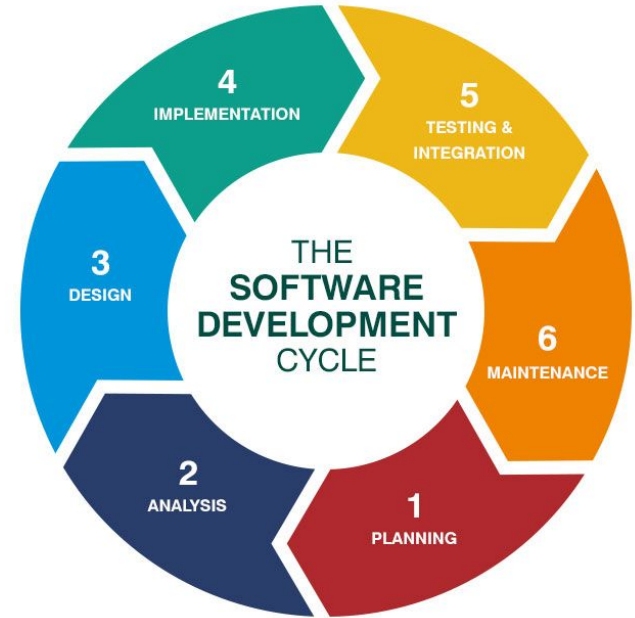
Computer architecture is essentially the blueprint of how a computer system is designed and how its various components interact to execute tasks. It is like the software we write and the hardware that actually does the work.

It is generally divided into three main categories:

- Instruction Set Architecture (ISA)
- Microarchitecture
- System Design

Importance In Software Development

Computer architecture acts as the physical foundation for the **SDLC**, directly influencing the **SRS document**. During requirements gathering, architectural constraints define non-functional requirements like performance, scalability, and hardware compatibility. In the design phase, software must be mapped to specific CPU and memory structures to ensure the final product meets these technical specifications efficiently.

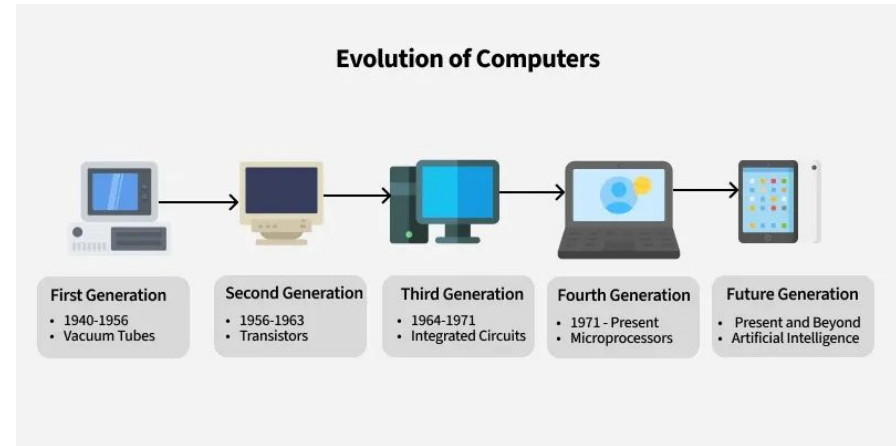


Basic Terminology

Key terminology includes the **CPU** (the "brain" executing instructions), **ALU** (handling arithmetic), and **Registers** (fastest internal storage). **RAM** provides volatile primary memory, while **Cache** reduces data latency. The **Instruction Set Architecture (ISA)** acts as the hardware-software interface, and **Buses** serve as communication pathways. Together, these components facilitate the **Fetch-Decode-Execute** cycle.

Historical Development of Computer Architectures

Early architectures evolved from the **Vacuum Tube** era (1st Gen) to **Transistors** (2nd Gen) and **Integrated Circuits** (3rd Gen), drastically reducing size and heat. The 1970s brought the **Microprocessor**, enabling personal computing. Evolution moved from simple **Sequential Processing** to modern **Parallel Computing, Multicore Processors**, and **Cloud Architectures**, prioritizing energy efficiency and massive data throughput.



RISC vs. CISC Architectures

RISC (Reduced Instruction Set Computer) focuses on simple, single-cycle instructions, prioritizing high clock speeds and efficiency, commonly seen in **ARM** processors. Conversely, **CISC** (Complex Instruction Set Computer) uses multi-cycle instructions that perform complex tasks in fewer lines of assembly, typified by **Intel x86**. RISC emphasizes software optimization, while CISC emphasizes hardware-level execution.

Pipeline Processing and Superscalar Execution

Pipelining speeds up execution by overlapping the stages of the Fetch-Decode-Execute cycle, allowing multiple instructions to be processed simultaneously like an assembly line. **Superscalar execution** goes further by using multiple execution units within a single processor to launch and complete several instructions per clock cycle. Both techniques significantly increase throughput without increasing the processor's raw clock speed

Computer Arithmetic and ALU Design

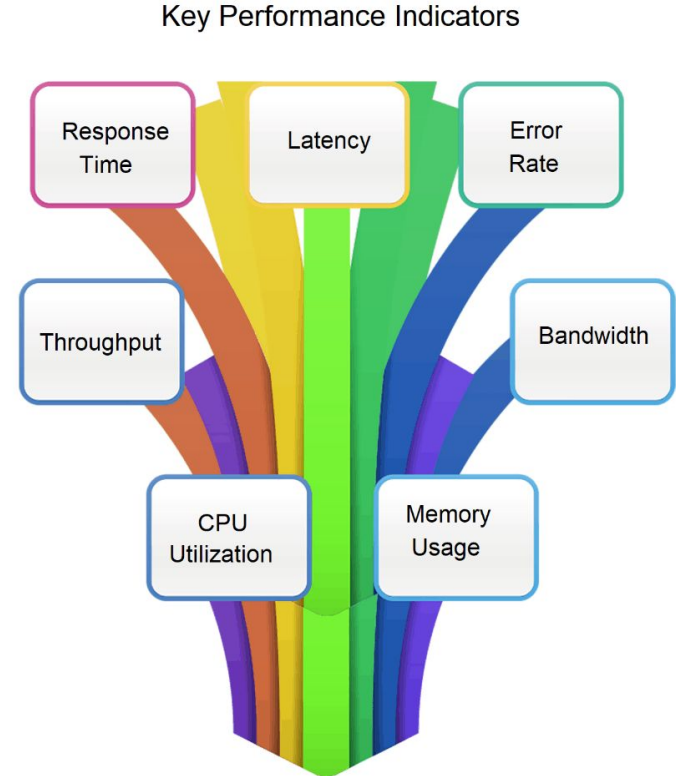
The **Arithmetic Logic Unit (ALU)** is the processor's core engine, performing binary operations like addition, subtraction, and bitwise logic. **Computer arithmetic** relies on fixed-point and floating-point representations (IEEE 754) to handle data. ALU design focuses on high-speed circuits, such as **Carry-Lookahead Adders**, to minimize propagation delay, ensuring the CPU executes mathematical instructions in the fewest possible clock cycles.

Virtual Memory and Paging Systems

Virtual memory allows computers to execute programs larger than physical RAM by using disk space as an extension. **Paging** divides this memory into fixed-size blocks called **pages**, which are mapped to physical **frames** via a **Page Table**. This system prevents memory fragmentation, enables efficient multitasking, and protects processes from accessing each other's private data.

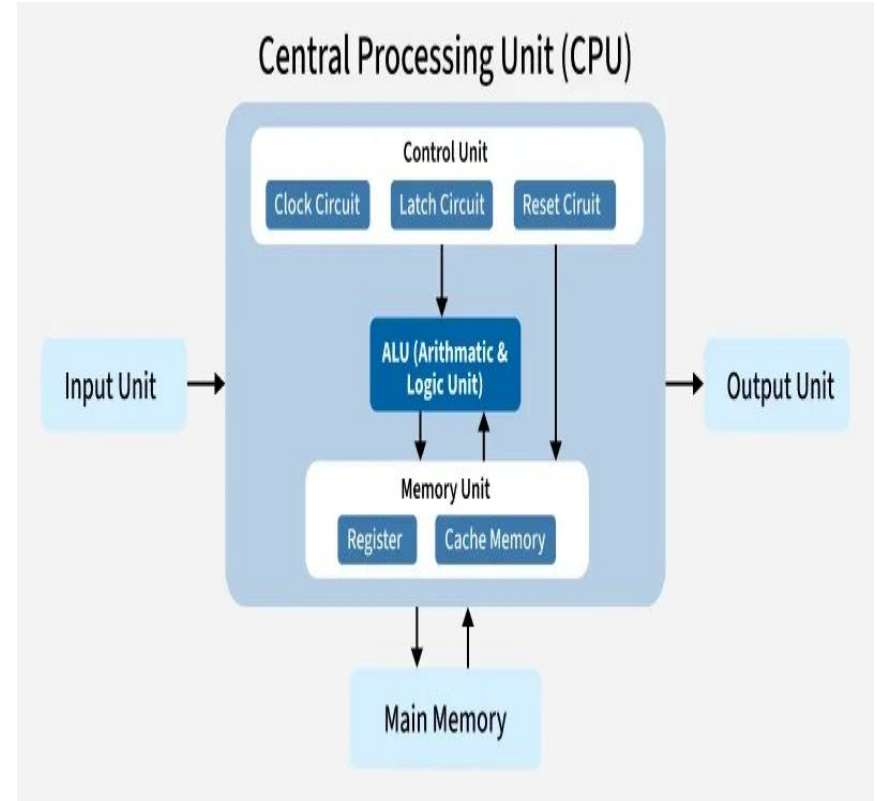
Benchmarking and Performance Metrics

Benchmarking evaluates a system's performance using standardized programs like **SPEC** or **Geekbench**. Key metrics include **MIPS** (Millions of Instructions Per Second), **FLOPS** (Floating-point Operations Per Second) for scientific tasks, and **Throughput** (total work per unit time). **Execution Time** remains the ultimate measure



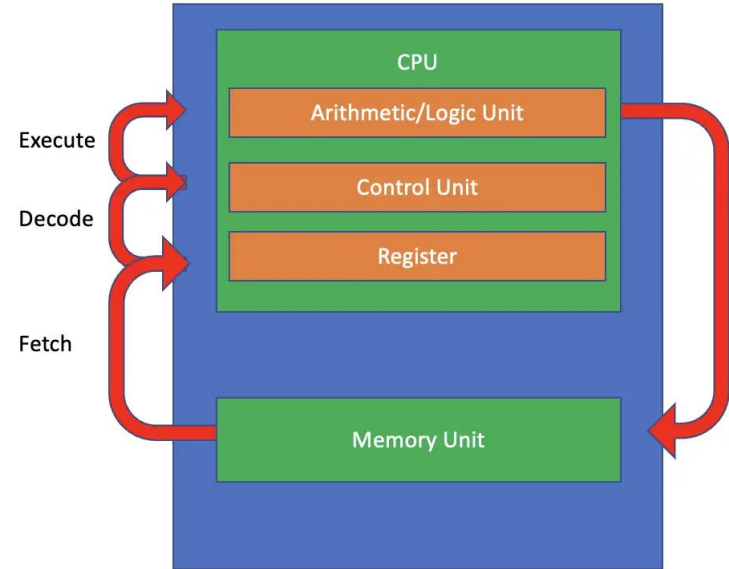
CPU Components and Functions

The **CPU** acts as the computer's brain, primarily consisting of the **Control Unit (CU)**, which directs data flow, and the **Arithmetic Logic Unit (ALU)**, which performs calculations. **Registers** provide immediate, high-speed storage for instructions, while the **Program Counter (PC)** tracks the next execution step. Together, they execute the **Fetch-Decode-Execute** cycle to process software commands.



How a CPU Executes Instructions

The CPU follows the **Fetch-Decode-Execute** cycle. First, it **fetches** an instruction from memory using the Program Counter. Next, the Control Unit **decodes** the instruction into signals. Finally, the **ALU** or execution unit performs the operation, such as addition or data movement. The result is then stored in a register or memory, and the cycle repeats.



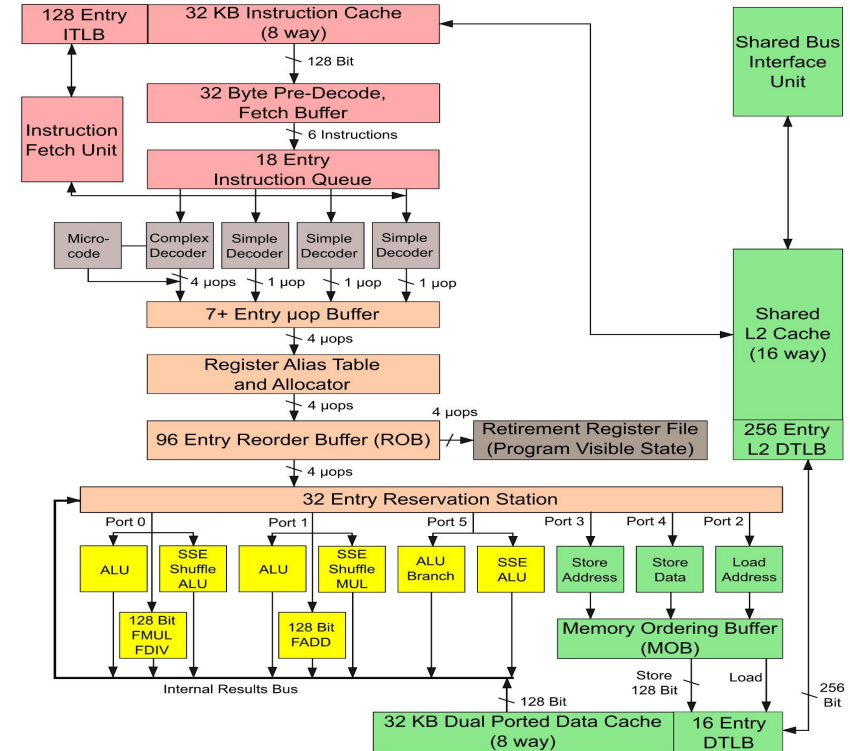
CPU Performance Factors

CPU performance is determined by three primary factors: **Clock Speed** (measured in GHz), which dictates how many cycles occur per second; **Instructions Per Cycle (IPC)**, reflecting architectural efficiency; and the **Instruction Count**, determined by the software and compiler. Together, these form the "Iron Law" of performance:

$$\text{Time} = \text{IC} \times \text{CPI} \times \text{ClockPeriod}$$

Microarchitecture Design

Microarchitecture represents the physical implementation of an **Instruction Set Architecture (ISA)**. It focuses on the internal design of the processor, including **pipelines**, **execution units**, and **cache hierarchies**. Key goals include maximizing **Instructions Per Cycle (IPC)** and minimizing latency. By organizing how data flows through gates and registers, microarchitecture determines a chip's power efficiency and raw computational speed

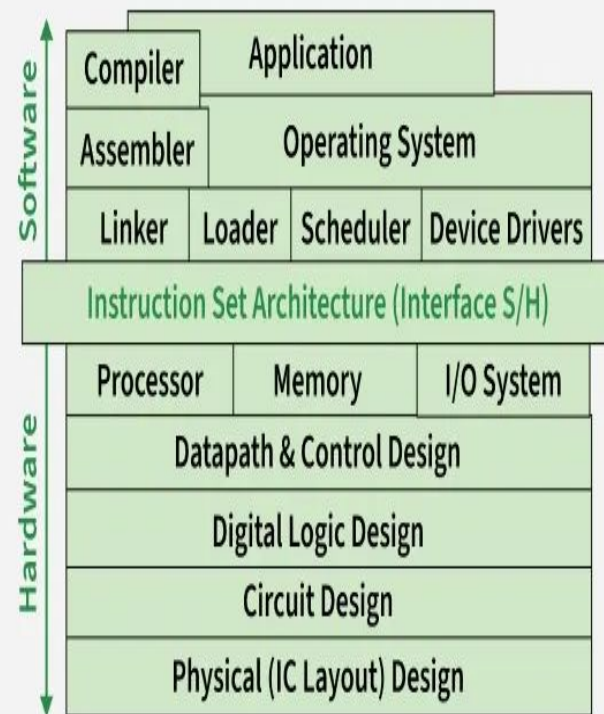


Intel Core 2 Architecture

Instruction Set Architectures (ISAs)

The **Instruction Set Architecture (ISA)** is the abstract interface between hardware and software. it defines the supported data types, registers, and the specific commands (like LOAD, STORE, or ADD) a processor can execute. Common examples include **x86** for desktops and **ARM** for mobile devices. The ISA ensures that software compiled for a specific "language" runs correctly on any compatible hardware.

Instruction Set Architecture (ISA)



Branch Prediction and Speculative Execution

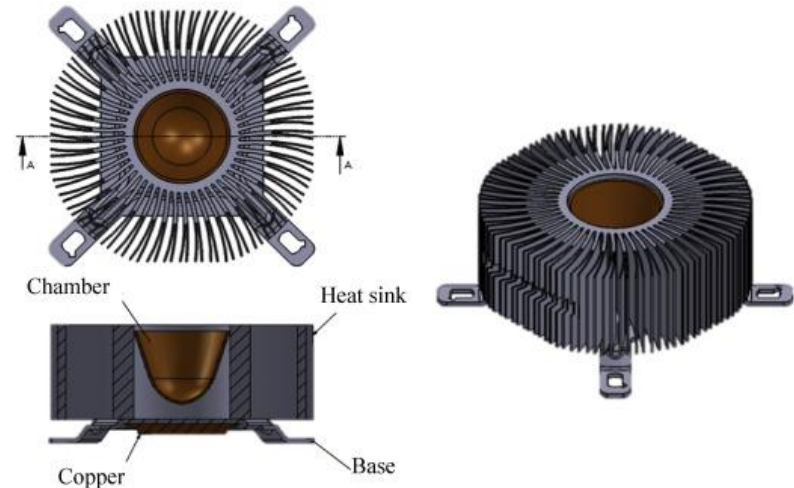
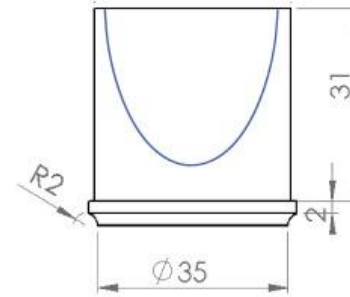
Branch prediction uses hardware algorithms to guess the outcome of conditional statements (like if-else) before they are resolved, keeping the pipeline full. **Speculative execution** then processes these predicted instructions in advance. If the guess is correct, performance jumps significantly; if wrong, the speculated work is discarded, and the pipeline restarts, causing a minor delay.

Parallelism at The Instruction Level

Instruction-Level Parallelism (ILP) allows a processor to execute multiple instructions from a single program simultaneously. By using **Pipelining**, different stages of instructions overlap, while **Superscalar** designs use multiple execution units to launch several instructions per clock cycle. Techniques like **Out-of-Order Execution** further boost ILP by running instructions as soon as their data is ready, rather than in their original program sequence

CPU Cooling and Thermal Management

- **Heat Sinks & Fans:** Use metal fins and airflow to pull heat away.
- **Liquid Cooling:** Circulates coolant to a radiator for superior heat dissipation.
- **Thermal Paste:** Fills microscopic air gaps between the CPU and cooler.
- **Throttling:** The CPU automatically slows down if it gets too hot.



*Thank
you!*